CSE450 : Capstone Project

Convo

Audio Watermarking & Confidentiality Chain in File Sharing

Team Members

Zaki Rehnoom Unmona	2105016
Debashri Roy	2105035
Anisa Binte Asad	2105036
Atika Tabassum Suchi	2105053
Fariha Ifrat	2105059

August 18, 2026

Contents

1 Audio Watermarking

1.1 Concept

1.1.1 Motivation and Problem Framing

Real-time communication platforms use transport encryption to secure audio while it travels across the network. However, this protection ends once the browser decodes the audio stream for playback. An authorized user can easily capture playback audio using local screen recorders, virtual audio devices, or external microphones—a security gap known as the *analog hole*.

Because web browsers run inside a restricted sandbox, platforms cannot install system-level recording blockers. Audio watermarking solves this problem by embedding a secret, participant-specific digital identifier directly into the audio stream right before it plays through the speakers.

If a participant records and leaks a meeting, this hidden identifier acts as a forensic signature. By scanning the leaked file, administrators can match the signature to the exact participant who leaked the audio, establishing accountability without requiring administrative software installations on client devices.

1.1.2 Core System Requirements and Trade-offs

Designing an audio watermarking system for real-time WebRTC conferencing requires balancing three main properties:

1. **Imperceptibility:** The watermark must remain completely inaudible so meeting participants hear clean, clear audio.
2. **Robustness:** The mark must survive real-world audio modifications, including lossy compression (such as WebRTC's Opus codec), microphone capture, and ambient room noise.
3. **Payload Capacity:** The amount of secret data hidden in the audio stream.
4. **Low Latency:** The embedding process must run in real time inside the browser without causing audio delays or stutter.

To meet real-time constraints, payload capacity is intentionally kept low. The system does not transmit large amounts of data; it only embeds a simple, highly redundant participant identifier (a unique random seed) that can survive aggressive lossy compression while remaining invisible to human ears.

1.1.3 Key Conceptual Principles

To make the watermark both invisible to human ears and durable against compression, the system relies on two core principles:

1. Psychoacoustic Masking (Achieving Imperceptibility) Human hearing does not perceive all sounds equally. When a loud sound plays at a certain frequency, human ears cannot detect quieter sounds playing at or near that same frequency.

The system analyzes meeting audio frame-by-frame to determine how much hidden signal can be added without being noticed:

- During loud speech, the system hides a slightly stronger watermark signal beneath the audio.
- During quiet pauses, the system scales the watermark down to near zero.

This ensures the watermark stays strictly below human hearing limits at all times, preventing static or distortion.

2. Spread Spectrum Modulation (Achieving Robustness) Simple watermarking methods (like changing low-level sample bits) fail under lossy audio compression like Opus, which intentionally discards fine sample details.

Instead, the system uses **Spread Spectrum** modulation:

- A secret, pseudo-random noise pattern is generated based on the participant's assigned seed.
- This noise pattern is spread thinly across all audio frequencies and over several seconds of time.

Because the signal is dispersed so broadly, compression algorithms treat it as faint background noise and leave it intact, allowing it to survive compression and microphone re-recordings.

1.1.4 Blind Forensic Detection

When a leaked recording is uploaded to the backend server, detection is performed *blindly*—meaning the detector does not need the original, un-watermarked audio file.

Detection follows a straightforward pattern-matching process:

1. The server regenerates the secret pseudo-random patterns for all registered meeting participants.
2. It compares the leaked recording against each participant's pattern over time.
3. Over a single fraction of a second, the hidden watermark is too weak to detect on its own. However, when measured over 5 to 10 seconds, random speech energy cancels out while the leaker's hidden pattern accumulates into a strong statistical match.

This allows the system to identify the source of a leak with high statistical confidence, even if the recording was compressed or captured using an external microphone.

1.2 Proof of Methodology

1.2.1 Grounding: Literature, Requirements, and Prior Art

The requirements and initial design direction were established via a preliminary literature review before implementation. Audio watermarking techniques split broadly into time-domain methods (e.g., LSB or echo hiding – simple but fragile against noise and compression) and transform-domain methods (e.g., DFT/DWT/DCT/SVD, particularly spread-spectrum (SS) approaches that disperse a watermark via a pseudo-noise sequence – more robust, at higher computational cost) *("Audio Watermarking: A Comprehensive Review," IJACSA, 2024).* A 2012 real-time audio watermarking study established spread-spectrum methods as the practical basis for low-latency operation, and a 2025 MDPI paper, *"Audio Watermarking System in Real-Time Applications,"* serves as the closest direct precedent: it combines time-domain spread spectrum with an MPEG-I Layer III-inspired psychoacoustic model at 256/512-sample frame sizes, achieving sub-3 ms latency and robustness

against lossy compression. Its core workflow – masking, spread-spectrum embedding, and verification via correlation – directly informed this project's architecture.

Four constraints, fixed prior to implementation, dictated every technical decision:
(1) **Imperceptibility** – the mark must remain inaudible;
(2) **Robustness** – it must survive Opus compression (WebRTC's default codec);
(3) **Low Latency** – watermarking must add single-digit millisecond latency inside WebRTC's real-time budget;
(4) **Efficiency** – it must run on client hardware with no GPU inside a single-threaded `AudioWorklet`.

1.2.2 From Planning to Implementation

The pre-implementation methodology originally specified a purely time-domain rule using an all-pole filter to approximate masking and a constant gain α. While latency-optimal, this indirect approximation proved unviable: without explicit per-critical-band analysis, no static filter shape stays inaudible across speech and silence while remaining strong enough to survive lossy codecs.

The shipped design retains spread-spectrum embedding and correlation detection but replaces the static filter with explicit per-band spectral analysis. By executing a 512-point FFT/IFFT pair per 256-sample hop, the system measures the exact masking threshold $T(b)$ of the frame directly. This trades a negligible latency cost (well within the 5.3 ms hop budget) for verifiable imperceptibility and robustness.

1.2.3 Pipeline at a Glance

Watermarking is done strictly on the client *playback* side. The mix bus in `audioWatermarkSetup.js` sums the local microphone with remote peer tracks into a single mixed stream. This mixed audio passes through an `AudioWorkletProcessor` (`audio-processor.worklet.js`), which embeds the watermark prior to speaker output. Consequently, any downstream capture mechanism (in-app recorders) captures the watermark, while outgoing peer streams remain pristine. Detection operates offline on the server: uploaded recordings are decoded to PCM and scored via correlation against candidate watermark configurations (`WatermarkDetectionService`, `WatermarkDsp`).

Two core architectural decisions depart from traditional substitutive schemes:

- **Additive spread-spectrum over bit substitution:** LSB substitution requires byte-exact sample preservation, which fails under lossy Opus encoding, WebRTC audio processing, and acoustic re-recording. Additive pseudo-noise (PN) patterns require only statistical correlation with a regenerated reference, making them significantly more durable.
- **Dynamic masking thresholds over static gain (α):** A constant gain factor is an unavoidable compromise between audibility in quiet passages and robustness in loud passages. A dynamic masking threshold ensures injection strength tracks the instantaneous perceptual capacity of the signal frame-by-frame.

1.2.4 Frame Constraints, Overlap-Add, and Masking Approximation

WebRTC delivers audio in 256-sample hops (≈5.3 ms at 48 kHz). A 256-point transform provides only ≈187 Hz bin resolution – too coarse for low-frequency Bark critical bands (≈100 Hz). To resolve this, analysis resolution is decoupled from output hop size: each 256-sample hop is analyzed within a 512-sample window ($N = 512$), yielding ≈94 Hz bin resolution.

To prevent phase discontinuities and boundary clicks between independently scaled frames, synthesis uses a 50% overlap Hann window within a constant-overlap-add (OLA) loop. Under linear summation, standard 50% overlap Hann windows satisfy the constant-overlap-add (COLA) property, guaranteeing a flat reconstruction gain envelope across frame boundaries. Output sample values are subsequently bounded via $y(t) = \text{clip}(x(t) + w(t), -1, 1)$. While saturation clipping is non-linear, operating within nominal signal headroom keeps saturation rare, ensuring the underlying COLA property prevents audible frame-seam artifacts.

Running a full MPEG Psychoacoustic Model 2 is computationally infeasible within a 5.3 ms real-time budget. The embedder instead computes a lightweight Bark-band spectral flatness measure (SFM) across 24 critical bands, using the geometric-to-arithmetic mean ratio of bin energies to distinguish tonal maskers from noise-like maskers. Masking thresholds are interpolated between empirical bounds ($\approx$18 dB offset for tonal content, $\approx$6 dB for noise), followed by a 3-tap adjacent-band spreading pass to model basilar membrane energy leakage.

1.2.5 Embedding and Bounded Search Cycles

For each frame f, a time-domain PN pattern is generated using a seeded PRNG, transformed via FFT, magnitude-reshaped to match the masking threshold $T(b)$ with a dB safety margin (g_{margin}), and inverse-transformed with Hermitian symmetry. The resulting time-domain watermark $w(t)$ is windowed and added to the host signal.

The PRNG seed itself is the mechanism for attribution, not just presence: the backend service (`WatermarkConfigService`) issues each meeting participant a distinct seed at session start, so every participant's playback stream carries a different, individually identifiable PN pattern. Detection therefore never asks only "is a watermark present" – it scores the recording against every seed registered for that meeting and reports whichever participant's pattern actually correlates, which is what turns the feature from tamper detection into leak attribution.

To prevent an unbounded alignment search during detection, frame PN patterns are periodic over an 8-second cycle length (H hops). The PRNG state for frame f depends strictly on $f \bmod H$. Because the PRNG state update operates as a linear counter, state evaluation at any arbitrary frame index executes in $O(1)$ time. This guarantees that any recording $\geq$8 s contains at least one complete cycle, bounding the detector search space to a single fixed period.

1.2.6 Blind Correlation Detection and Drift Correction

Detection operates without the original un-watermarked audio. The detector regenerates the candidate watermark signal $r(t)$ by applying identical masking analysis to the received recording $a(t)$, then scores candidate alignments using per-frame normalized correlation:

$$c_f = \frac{\sum_i a_i r_i}{\sqrt{\left(\sum_i a_i^2\right)\left(\sum_i r_i^2\right)}} \in [-1, 1].$$

Per-frame normalization prevents loud audio passages from artificially dominating the overall correlation score. To assess final confidence, per-frame scores are combined via energy-weighted aggregation:

$$C_{\text{final}} = \frac{\sum_f \left(\sum_i a_i r_i\right)}{\sum_f \sqrt{\left(\sum_i a_i^2\right)\left(\sum_i r_i^2\right)}}.$$

Search Optimization and Out-of-Sample Validation. Candidate parameters (seed, phase, cycle position) are evaluated in a two-stage coarse-to-fine search over the initial ≈0.85 s of the recording. For acoustic re-recordings, where physical clock discrepancies cause sample drift over time, the detector re-locks alignment at 1-second intervals via dot-product sliding correlation against the synthesized watermark.

To eliminate selection bias (the look-elsewhere effect caused by evaluating thousands of hypotheses), the candidate alignment selected during search is evaluated for reporting on a *held-out* segment of the recording. As an auxiliary duration-invariant metric, the detector computes $\text{mean}(c) \cdot \sqrt{n}/\text{std}(c)$ over n scored frames, which behaves approximately like standard normal noise under the null hypothesis while scaling with $\sqrt{n}$ under true detection.

Because every registered participant is scored against the same recording, attribution requires more than the winning user simply scoring highest: a user is only reported as detected when their held-out score clears an absolute detection threshold *and* exceeds the runner-up's score by a minimum margin. Without that margin check, two close, noise-driven scores could flip which participant gets named essentially at random.

1.2.7 Empirical Findings from Prototyping

Key architectural choices were validated by failure modes observed in early single-frame, constant-α prototypes:

- **Single-frame detection instability:** Single-frame correlation variance on random audio signals is comparable in magnitude to the embedded signal strength. Multi-frame correlation aggregation is mathematically necessary to lower the noise floor.
- **Codec attenuation:** Opus re-encoding attenuated unshaped, flat-spectrum PN sequences by 80–85% in prototype testing. Shaping injection energy to match the host signal's masking threshold protects the mark from codec noise-suppression algorithms.
- **AEC/NS interference:** WebRTC echo cancellation and noise suppression heavily suppress broadband additive noise – a problem in the early prototype, which watermarked the outgoing microphone capture directly. The shipped design instead watermarks the already-mixed *playback* signal (see *Pipeline at a Glance*), so embedding happens strictly after this browser's own AEC/NS has already run on the raw microphone input rather than before it – though an external recorder's independent AEC/NS during acoustic recapture remains outside this project's control.

1.2.8 Why the Mark Survives What It Must, While Staying Inaudible

Imperceptibility and robustness follow from the same design, not separate mechanisms. Each frame's injected energy is capped at a safety margin below *that frame's own* masking threshold $T(b)$, so no single frame is ever audible on its own. Robustness instead comes from the classical spread-spectrum processing-gain argument that underlies DSSS communications: watermark energy is spread thinly across the full frequency band and across many independent frames, and the detector coherently integrates correlation over all of them via the energy-weighted aggregate C_{final}, so effective detection SNR grows with the number of frames scored even though per-frame SNR is deliberately kept at or below the threshold of audibility. This is what lets the mark survive Opus's lossy re-encoding, this browser's own AEC/NS chain, and even a full acoustic round-trip through a speaker and an unrelated microphone: none of these need to preserve the signal at the sample

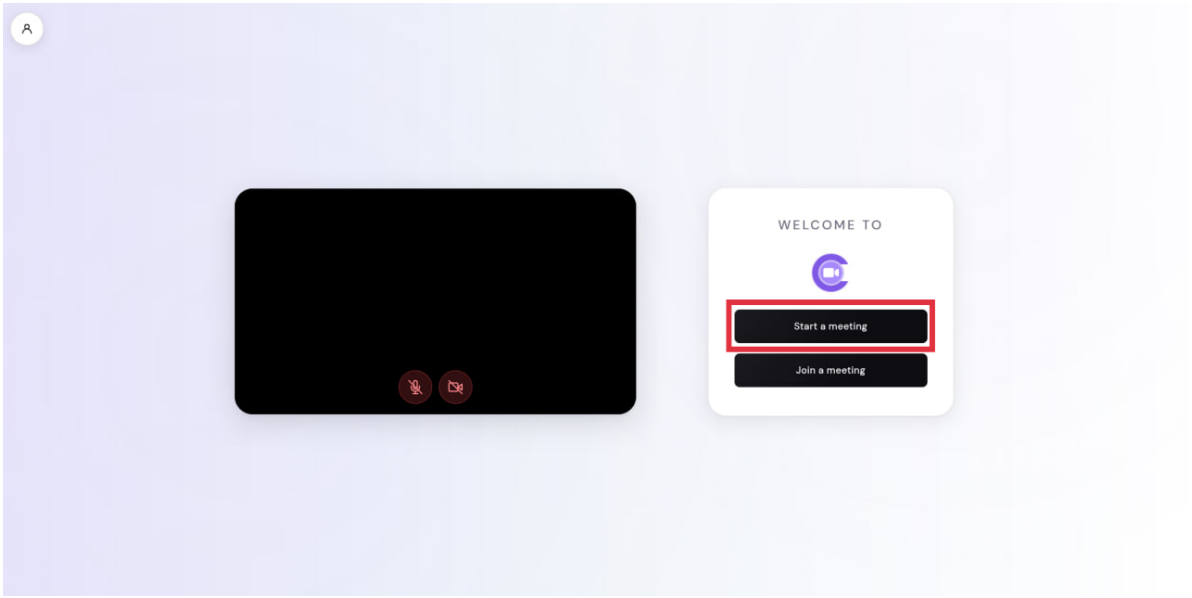
level, only enough aggregate correlation across hundreds of frames for C_{final} to clear the detection threshold – exactly the property additive spread-spectrum was chosen for over substitutive insertion in the first place.

1.3 How to Test

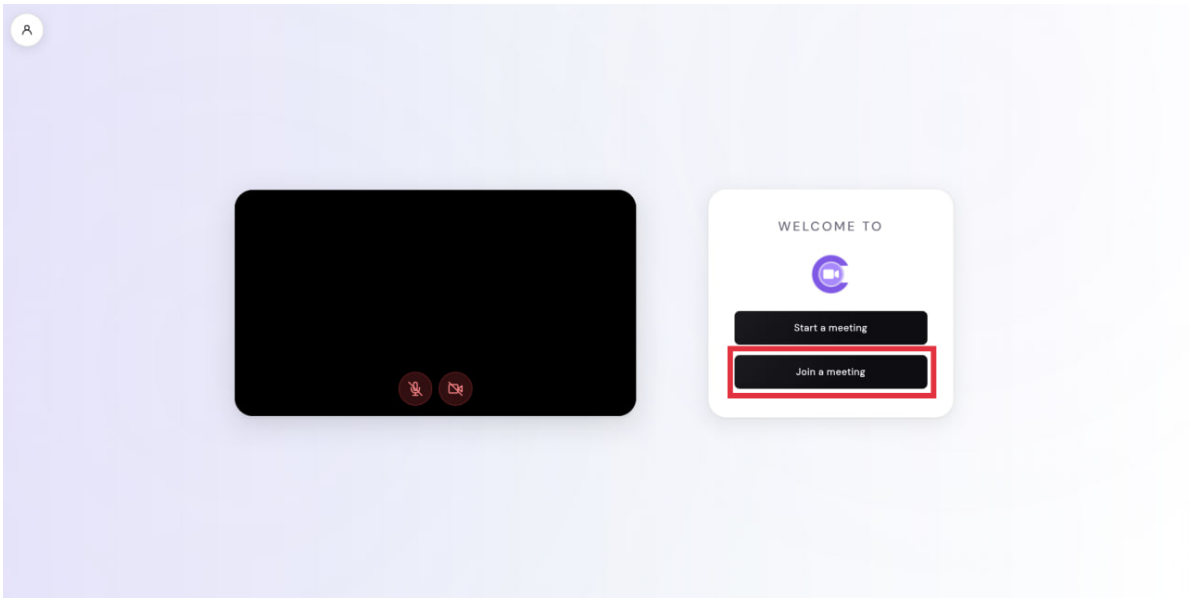
You will need two people to try this – one to talk in the call, and one to receive the recording afterwards. Two browser tabs or two devices both work.

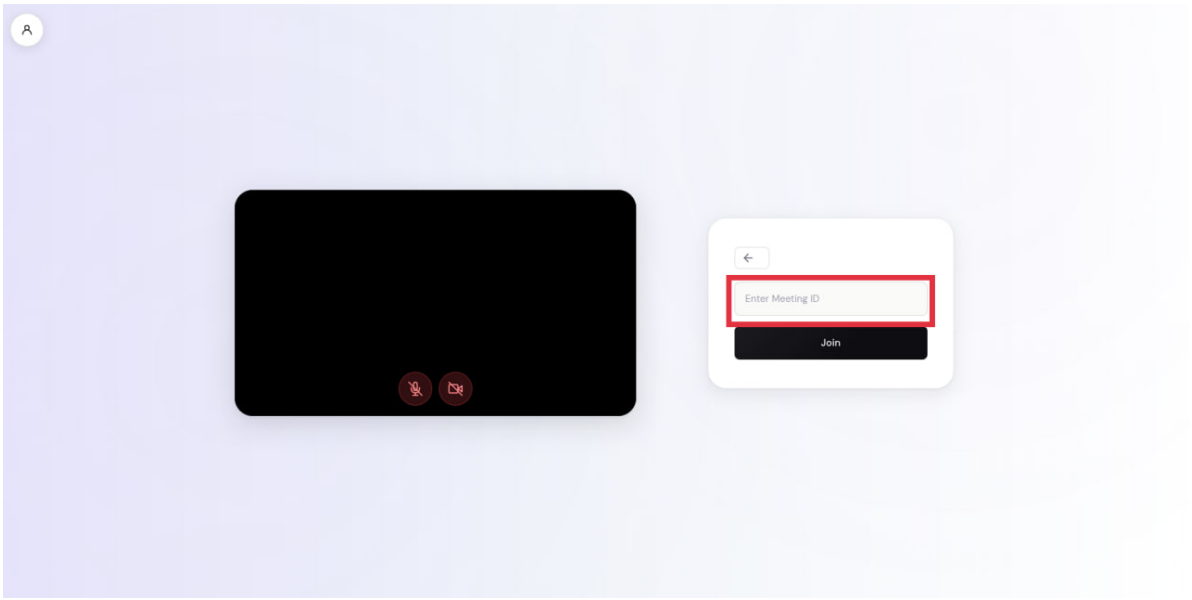
Part A – Record a call and check the mark.

1. Open the web app and click **Start a meeting**. A Meeting ID and a Meeting Link will appear on screen.



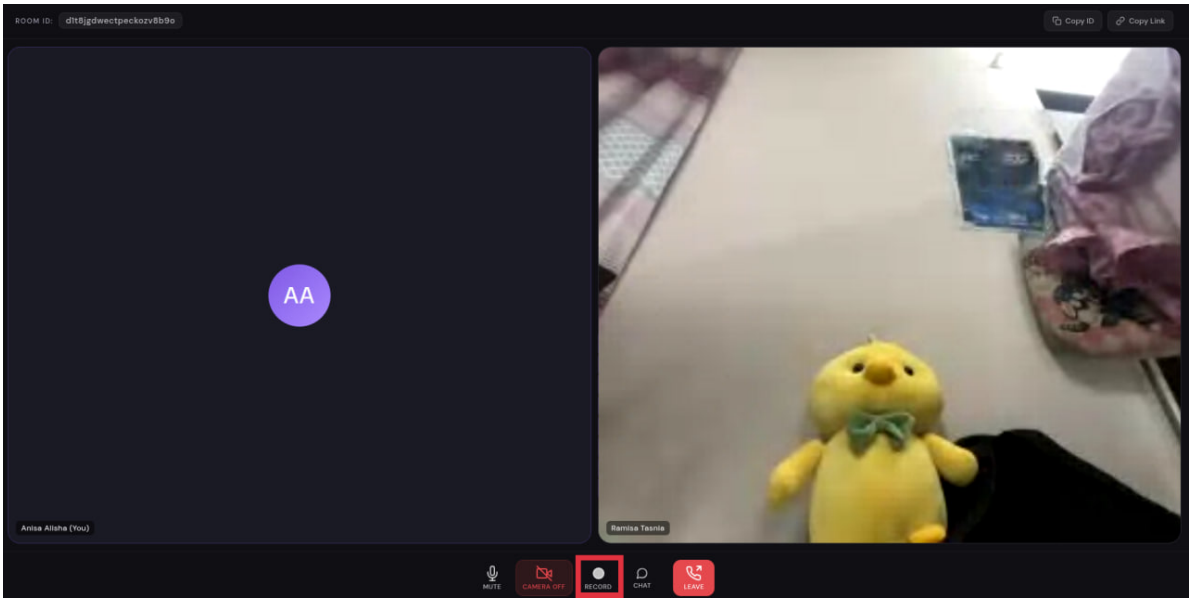
2. On a second device (or a second browser tab), click **Join a meeting** and type in the same Meeting ID, or just open the Meeting Link.



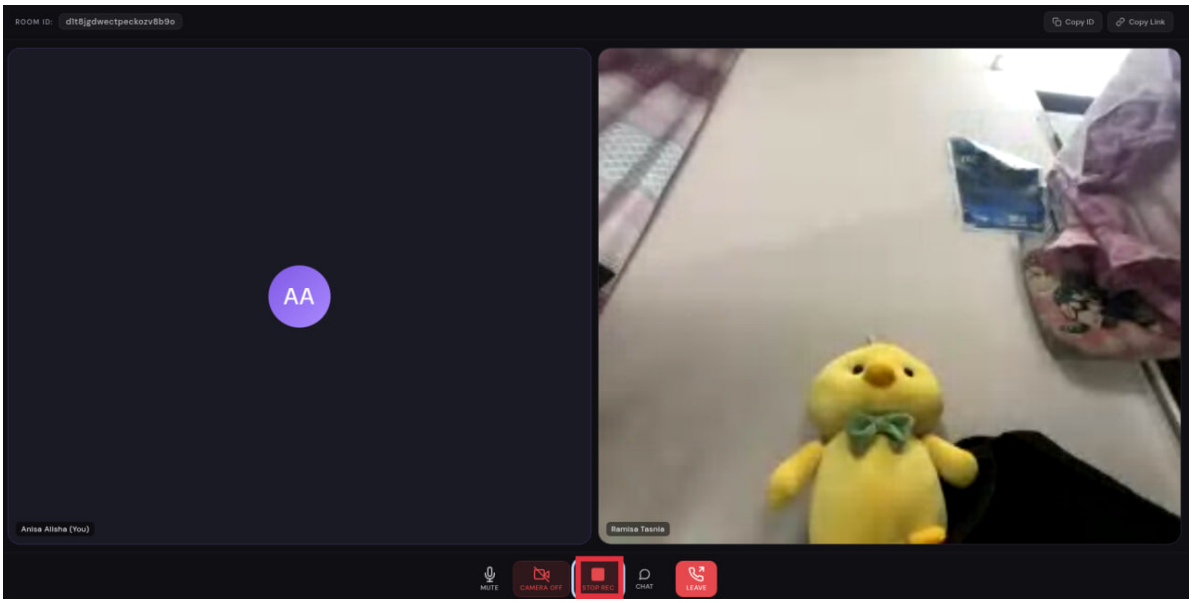


3. Once both sides can see and hear each other, start talking.

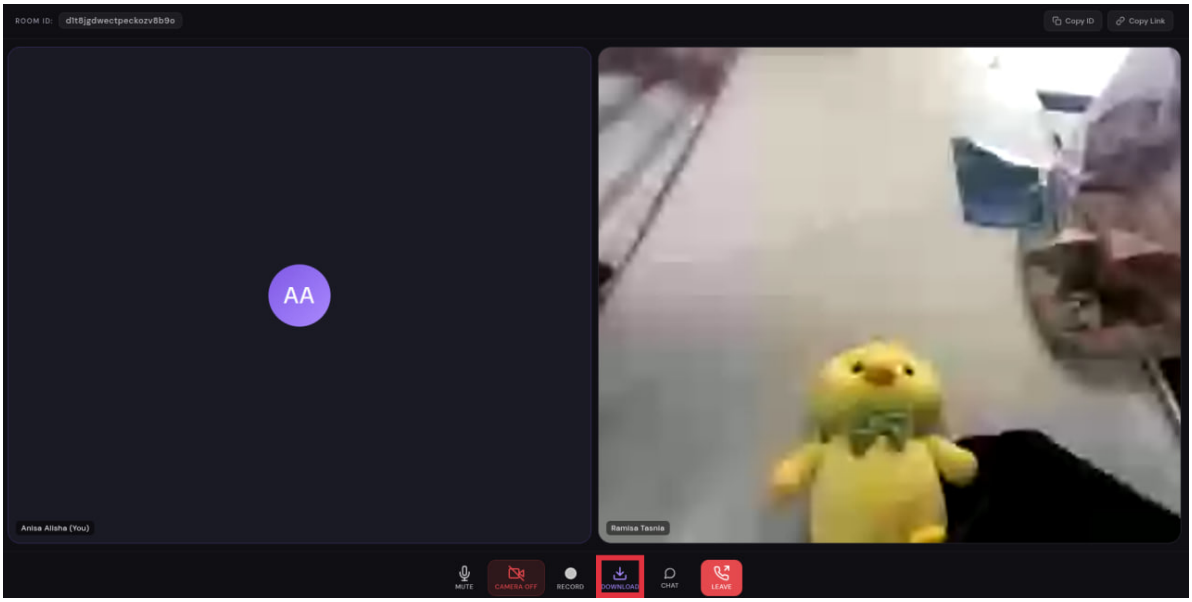
4. On the toolbar at the bottom of the call, click **Record**. The button changes to **Stop Rec** while it's recording.



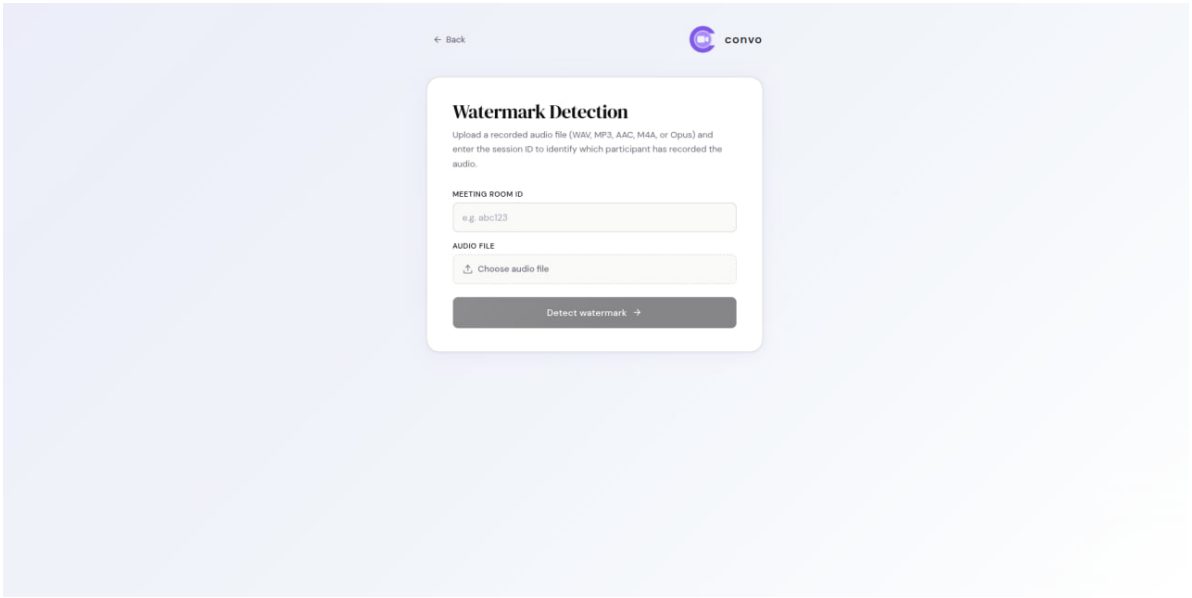
5. After you've spoken for a bit, click **Stop Rec**.



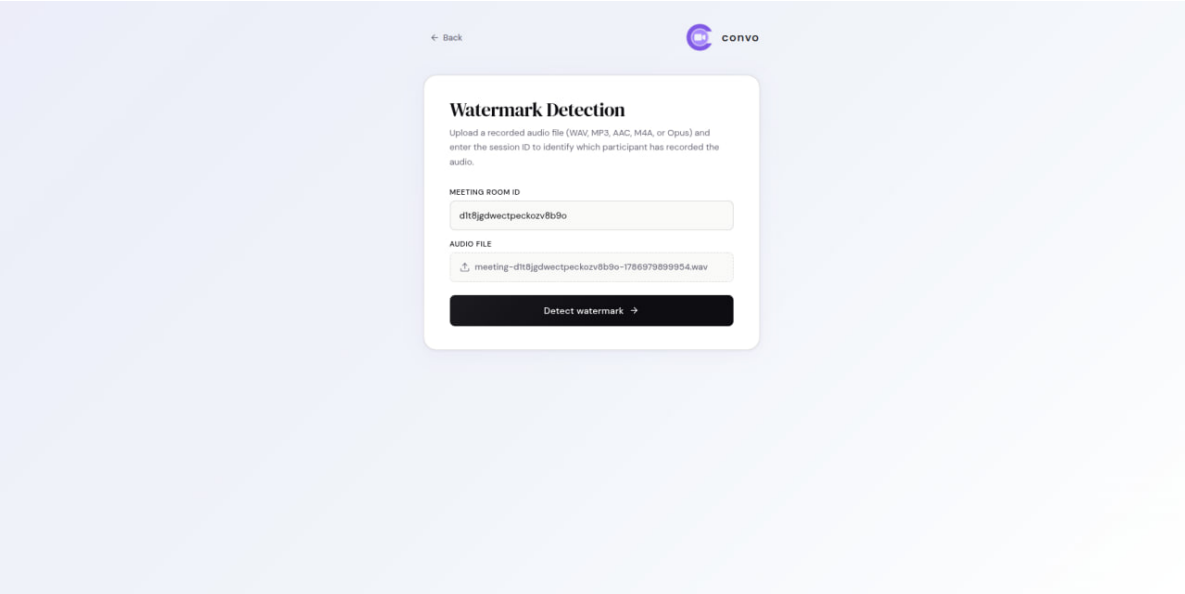
6. A **Download** button appears on the toolbar. Click it – a short audio file is saved to your computer.



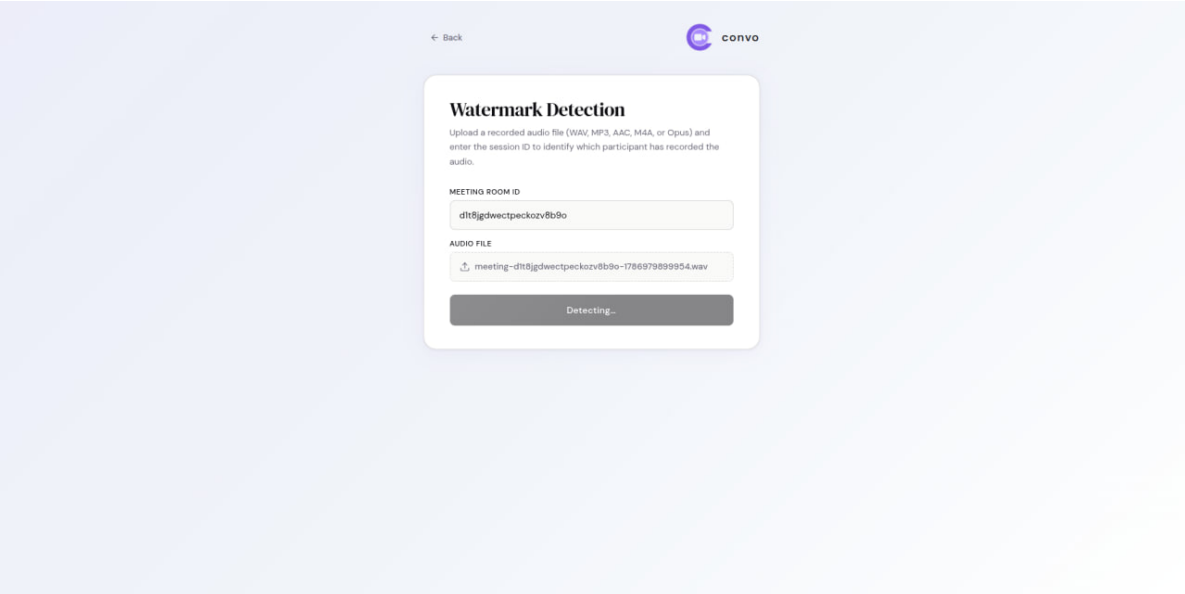
7. Open a new browser tab and go to the watermark-test page.



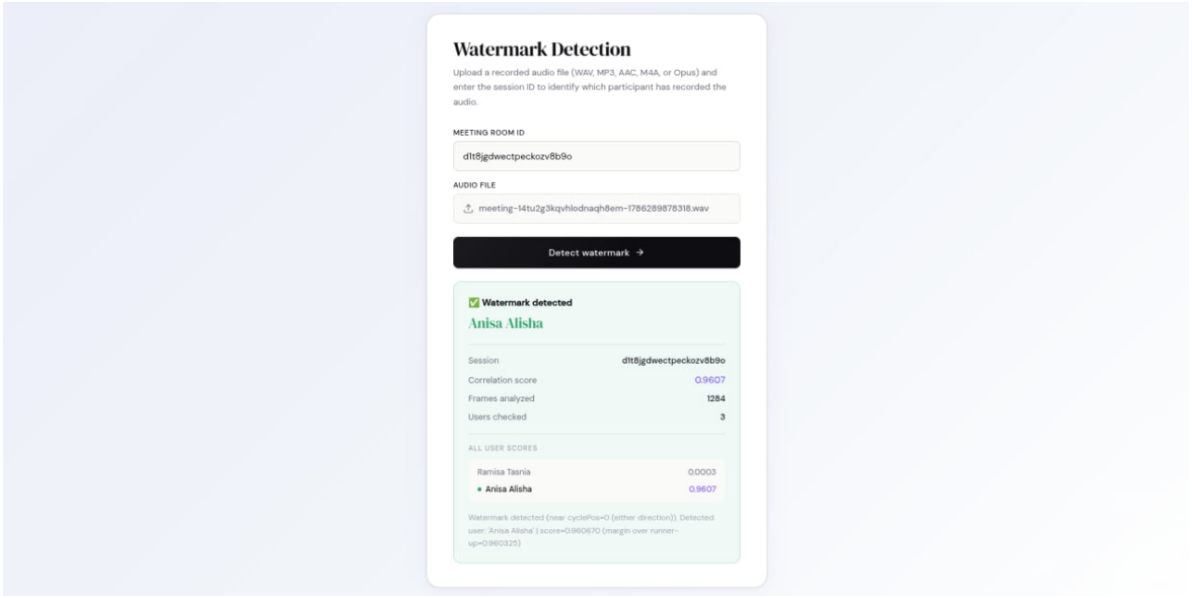
8. Type the same Meeting ID from step 1 into the **Meeting Room ID** box.
9. Click **Choose audio file** and select the file you just downloaded.



10. Click **Detect watermark** and wait a few seconds.



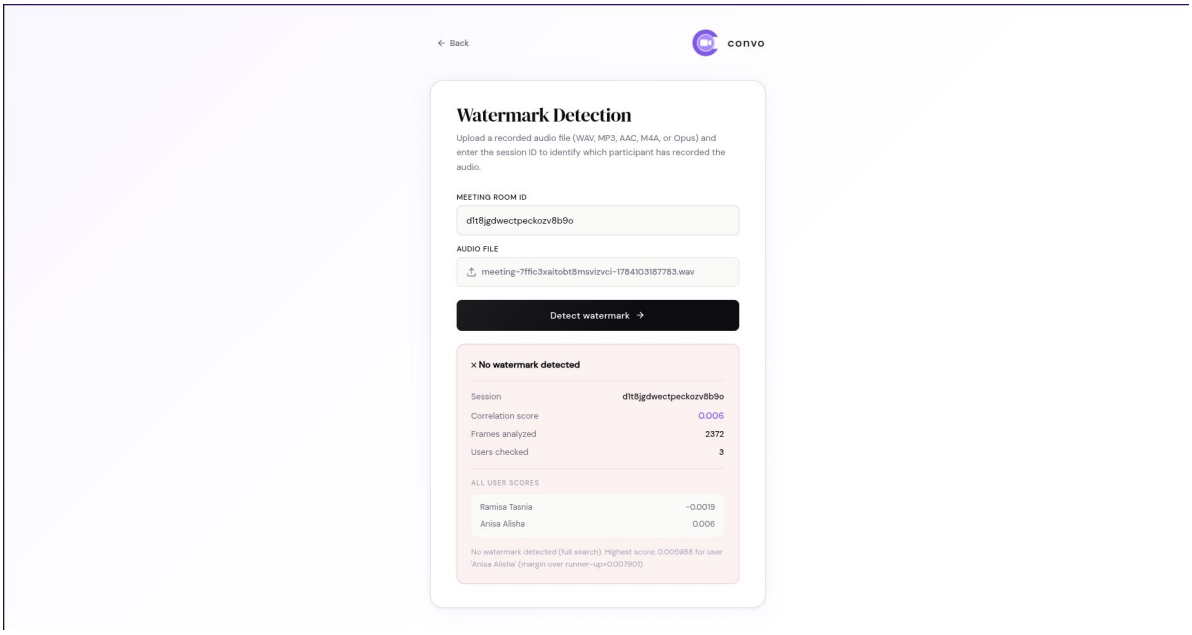
What you should see: a green **Watermark detected** result, naming which participant's voice it matched. Nothing about the call sounded different while you were talking, but the recording still carried an identifying mark all the way through – that's the feature working.



Part B – Confirm it isn't just saying yes to everything.

1. On the same watermark-test page, upload a completely unrelated audio file (anything not from this call), or type in a Meeting ID that was never used.
2. Click **Detect watermark** again.

What you should see: a red **No watermark detected** result. This confirms the check is a genuine check, not one that always reports success.



2 Confidentiality Chain in File Sharing

2.1 Concept

The Confidentiality Chain in File Sharing guarantees two properties for every file exchanged within a Convo meeting: that its content remains confidential to unauthorized parties, and that its origin and integrity can be verified at every point it is shared or re-shared. This is achieved by combining end-to-end encryption with a cryptographically signed provenance chain, addressing two distinct classes of threats.

Outsider Attacks

An outsider is any party outside the session who may still intercept network traffic.

Threat	Mitigation
Intercepting the file in transit and reading its contents.	End-to-end encryption: a one-time content key encrypts the file using AES-GCM, and is itself encrypted per recipient using an ECDH-derived key. Intercepted traffic yields only ciphertext.
Querying the backend directly for another session's file metadata or keys.	Row-level security policies restrict metadata/key access to verified participants of that specific session.

Insider Attacks

An insider is an authorized participant who may misuse that access.

Threat	Mitigation
Modifying a file or its metadata after sharing, then presenting it as unmodified.	Every transfer carries an ECDSA-signed block binding the file's content hash and metadata to the sender. Any modification invalidates the signature.
Re-sharing a file to obscure its origin, or passing off a flagged file as new content.	Each signed block links to its predecessor (`previousHash`), forming a verifiable chain. Before forwarding, the sender's client re-verifies this chain and refuses to send if it fails.
Forwarding a file to a party outside the original authorized recipients.	Encryption keys are derived per recipient at share time; an unauthorized party has no matching private key and cannot decrypt a forwarded file.

2.2 Proof of Methodology

2.2.1 Grounding: Requirements and Prior Art

The confidentiality chain sits on top of an existing, unencrypted capability: files could already be dragged into a meeting's chat and streamed peer-to-peer over an `RTCDataChannel`, whose DTLS transport is confidential to network eavesdroppers but gives none of the application-layer guarantees the Concept section's threat model calls for. Rather than a formal literature review, the design direction here follows three well-established, independently verifiable protocols: *envelope encryption* (a random per-message content key encrypts the payload once, then that key – not the payload – is re-encrypted individually per recipient), the pattern used by S/MIME, PGP, and every major E2EE messaging protocol to avoid re-encrypting a large payload once per recipient; *Diffie–Hellman key agreement over elliptic curves* (ECDH-P256, as used in the Signal Protocol's X3DH and in TLS 1.3) to derive a shared secret without either party's private key ever crossing the network; and *content-addressing* (identifying data by a hash of its own bytes, as Git and IPFS do) to give a file a stable identity that is independent of which meeting, sender, or filename it was shared under.

These map directly onto four constraints, fixed prior to implementation, that dictated every technical decision below:

- **Confidentiality** – unreadable to anyone but the sender and the intended recipients, including the backend itself.
- **Integrity and Attribution** – any tamper, however small, is detectable, and every file is cryptographically tied to the identity that actually sent it.
- **Provenance Continuity** – history stays traceable and verifiable across re-shares and across meetings, not just within whichever browser tab happened to witness it.
- **Zero-Trust Backend** – the server never receives plaintext or key material, and nothing it is asked to vouch for is accepted from a client without independent re-derivation or database enforcement.

2.2.2 From Planning to Implementation

The chain-linkage logic was originally implemented and verified entirely client-side: each browser tab kept an in-memory `Map` from a file's signed hash to its signed block (`createChainStore` / `reconstructChain`), and an incoming file's `previousHash` was resolved by looking that map up locally. This was latency-optimal – no network round trip needed to check linkage – but it silently assumed that whatever chain history a tab happened to have witnessed was the whole truth. It was not: a file's chain almost always spans multiple meetings, and no single browser tab is present for all of them. The result was two symmetric failures observed directly in testing – nicknamed the "Meeting A $\rightarrow$ Meeting B bug" in the code's own history – where a file forwarded into a second meeting was flagged *chain broken* the first time (the receiving tab had never seen the prior hop, so the local map came up empty), and then, confusingly, reported as *verified* on a later retry once that block happened to get cached locally, even though nothing about the file's actual, durable history had changed.

The shipped design keeps the same signed-hash-chain structure but replaces local reconstruction with a query against the backend's durable `transfer_metadata` table (`GET /api/transfer/metadata/history/{contentHash}`), re-verified with a full ECDSA signature check on every hop rather than trusted on sight. This trades one network round trip per verification for a chain-linkage

result that is correct regardless of which meetings a given browser tab was or wasn't present for – the property the local-map version could never actually provide.

2.2.3 Pipeline at a Glance

Sending a file, orchestrated by `prepareFileForTransfer` in `provenancePipeline.js`, runs a fixed sequence of stages, each of which the sending UI (`ChatFileShare.jsx`) surfaces as live progress:

1. The raw file bytes are hashed on their own (`computeContentHash`) to obtain a *content hash* – a content-addressed identity for this exact file, independent of who is sharing it or in which meeting.
2. That content hash is used to ask the backend for any prior transfers of this same content, and if one exists, its most recent entry is re-verified (signature and all) before being trusted as this transfer's `previousHash` – a corrupted or tampered prior entry causes the send to refuse to link to it (`ProvenanceLinkError`) rather than silently starting a fresh, disconnected chain.
3. A metadata block (`transferId`, session, sender, filename, size, MIME type, `previousHash`) is POSTed to the backend, which mints the `transferId` and timestamp itself – a client-supplied clock is never trusted.
4. Each recipient's ECDH-P256 public key is resolved, preferring the live session key that recipient announced over the meeting's data channel when it opened (the "session-key handshake") over their globally registered key, so encryption targets the exact device present in the room rather than an arbitrary one of that user's devices.
5. A signing hash is computed over the file bytes plus the canonicalized metadata block, then signed with the sender's ECDSA-P256 private key – which never leaves the browser's `IndexedDB` – and the resulting `fileHash`/`signature`/`contentHash` are PATCHed back to the backend, completing the pending metadata row.
6. The signed block is embedded as a small self-describing header directly in front of the raw file bytes.
7. The whole wrapped buffer is envelope-encrypted – one random AES-256-GCM content key encrypts it once, and that content key alone is re-encrypted per recipient under an ECDH-derived shared key – and the resulting ciphertext is chunked and streamed peer-to-peer over the existing `RTCDataChannel`.

At no point does the backend receive a file byte, a content key, or a private key; it only ever stores the small provenance row (hashes, signature, sender, session, timestamp) needed to answer future chain-history queries.

Receiving a file mirrors this in strict order via `verifyIncomingTransfer.js`: decrypt the envelope, unwrap the header, then verify – cheapest check first.

1. The file hash is recomputed from the received bytes and metadata and compared against the signed `fileHash` – catching any bit-level corruption or tampering first, before spending a network round trip on anything else.
2. The declared `previousHash` is then resolved against the backend's durable chain history, never a local cache.

3. The signature is verified against the sender's *full* registered key history, not just their current key.
4. Only once every one of those checks has passed is the sender's display identity resolved and shown.

Any lookup failure along the way is treated as a failed verification, never as a silent pass.

2.2.4 Canonicalization, the Two Hashes, and the Wire Format

Two SHA-256 hashes are computed over the same file for two different, deliberately unmixed purposes:

- **Content hash** – the digest of the raw file bytes alone, nothing else. It exists purely to give identical file content a stable identity across every session it is ever shared in, independent of sender, filename, or timestamp; it is what chain-root uniqueness and cross-session lookups are keyed on.
- **Signing hash** (referred to as `fileHash` in the metadata) – the digest of the file bytes concatenated with the *canonicalized* metadata block. This, never the content hash, is what actually gets ECDSA-signed. Mixing metadata into the signed hash is what cryptographically binds a specific sender, session, and `previousHash` to a specific set of file bytes; without it, a signature over the file alone could be silently replayed onto a different metadata context.

Canonicalization is a cross-language contract, not an implementation detail either side owns unilaterally: `canonicalize.js` on the frontend and `Canonicalizer.java` on the backend must produce byte-identical JSON for the same metadata block, or every signature verification fails silently despite both sides agreeing on the underlying data. The rules:

- A fixed key order (alphabetical, pinned explicitly rather than derived from a sorted map, so a future field that breaks alphabetical order doesn't silently reorder the output).
- No incidental whitespace in the serialized JSON.
- `previousHash` is always emitted as an explicit JSON `null` for a root transfer rather than omitted – an omitted key and an explicit null are different byte strings, and only one of them is what the other language actually produces.

The signed block (metadata plus `fileHash` plus `signature`) is embedded ahead of the raw file bytes behind a minimal 9-byte binary header: a 4-byte magic value, a 1-byte version, and a big-endian 4-byte length for the JSON block that follows. The header is deliberately self-describing and versioned so a future format change, or a genuinely malformed buffer, fails loudly with a specific parse error rather than being silently misinterpreted as a differently-shaped payload.

2.2.5 Chain Linking and Atomic Root Claiming

Every transfer after the first for a given file content declares the `fileHash` of the transfer it was shared from as its own `previousHash`, forming a signed, backward-linked chain rooted at that content's very first share. Establishing who gets to be the root matters: if two people share the exact same fresh file content for the first time at nearly the same moment, only one of those shares should become the chain's root, with the other correctly recognized as content that already has history.

- **The problem.** The original find-then-check logic for this raced under concurrency – two simultaneous first-shares of identical content could both query an empty history and both believe themselves to be the root, producing two disconnected root entries for content that is actually one chain.
- **The fix.** The shipped design instead makes `content_hash` the primary key of a dedicated `chain_roots` table and claims a root by inserting into it; the database's own uniqueness constraint, not a SELECT the application runs first, is what makes "one root per content hash" hold under concurrent requests, and the losing writer gets a constraint violation it turns into an actionable error rather than a silent duplicate root.
- **The side benefit.** This same mechanism doubles as an anti-laundering check for free: content that was ever shared before already holds this row, so a later attempt – malicious or not – to reintroduce it as a brand-new, unlinked root hits the identical conflict.

2.2.6 Envelope Encryption and Non-Destructive Key Rotation

Two design choices make this both confidential and resilient to key changes:

- **Separate keys per purpose.** Every user carries two independent P-256 keypairs: an ECDSA keypair for signing (attribution and integrity) and a separate ECDH keypair for key agreement (confidentiality). Reusing one asymmetric key for both roles is a well-known cryptographic hazard the design avoids by construction.
- **Additive key registration.** Keys are generated and held per-device in the browser's `IndexedDB` and registered with the backend additively: registering a new key never overwrites or deletes an old one. A signature made with a key a user has since rotated away from – by clearing browser storage, or by using a new device – remains verifiable forever after, because verification tries every key the sender has ever registered (`verifyBlockWithHistory`), not only their current one.

Confidentiality itself is envelope encryption, not per-recipient re-encryption of the whole payload:

- A single random AES-256-GCM content key encrypts the wrapped provenance buffer exactly once, regardless of how many recipients a file is being broadcast to in a group meeting.
- That content key is then individually wrapped under an ECDH-derived shared key per recipient, each derivation combining the sender's private key with one recipient's public key so the shared secret itself never crosses the network.

A file sent to ten meeting participants is therefore encrypted once and key-wrapped ten times, not encrypted ten times.

2.2.7 Durable Verification and Point-in-Time Authorization

Trust in a chain hop is never taken on the word of the hop's own metadata. Two independent checks gate it:

- **Signature re-verification** against the sender's full key history, as described above.
- **Authorization** against `session_participants` – was the declared sender actually present, at the time, in the specific meeting where this hop claims to have happened?

The authorization check is deliberately evaluated against the session the hop itself occurred in, not the chain's original founding session; an earlier version checked every hop against the file's very first meeting, which meant a legitimate later forward could fail authorization simply because its sender wasn't present at the chain's original creation, even though they were correctly present in the meeting where they actually received and re-shared the file. `session_participants` is explicitly a point-in-time attendance snapshot, not a live permission list – it does not reflect access later revoked – a known and documented scope boundary rather than an oversight. Every lookup on this path fails closed: a network or authorization-service hiccup is treated as *unauthorized* or *unverified*, never silently passed through as valid.

2.2.8 Empirical Findings from Prototyping

Several concrete bugs, not just theoretical risks, shaped the shipped design:

- **Cross-session verification cannot be local.** As described above, an in-memory chain reconstruction produced both false "chain broken" and false "verified" results for the same file depending on which meetings a given browser tab happened to be open for, because there was no durable, shared source of truth to check against.
- **Concurrent first-shares need a database-enforced invariant, not an application-level check.** The original find-then-check root claim raced and could mint two disconnected roots for one piece of content; moving the invariant into a primary-key constraint closed the race and, as a side effect, provided the anti-laundering property.
- **Authorization must be checked against the hop's own session, not the chain's origin.** Checking every hop against `originSessionId` instead of that hop's actual `sessionId` rejected legitimate re-shares whose sender simply wasn't present at the chain's original creation.
- **Key rotation must be additive.** An earlier upsert-style key registration overwrote a user's stored key on each new registration, which stranded every signature made with the prior key the moment a device regenerated its keypair; switching to additive registration plus history-aware verification made rotation non-destructive.

2.2.9 Why Confidentiality and Provenance Hold Together

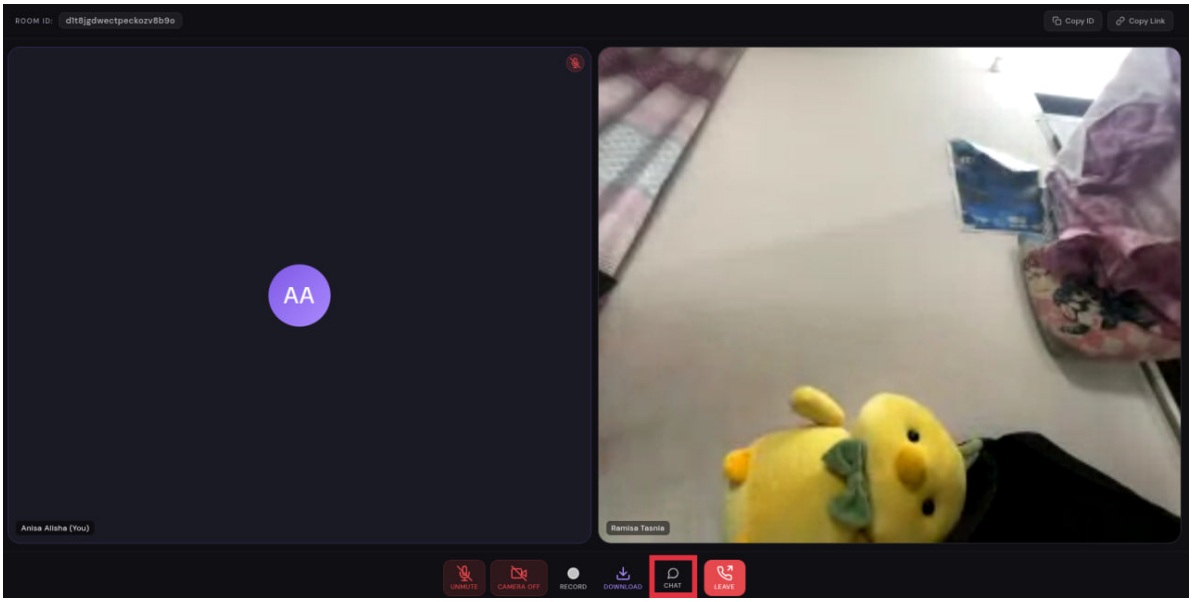
The two guarantees this section opened with – that only the intended recipients can read a file, and that its history can be independently verified – come from the same architectural choice rather than two separate mechanisms bolted together. The backend is only ever handed content-addressed hashes and signatures, never plaintext, a content key, or a private key, which is what makes it possible to trust it as the durable backbone for chain history without having to trust it with confidentiality at all. The identical canonicalization contract that lets a receiver detect a single altered byte in a file is the same contract that lets a completely different session, verifying a hop it has never seen before, reconstruct exactly the bytes the original sender signed. And the same content hash that keys chain-root uniqueness at the database layer is what lets provenance be verified across meetings the verifying client was never a part of, without ever trusting anything cached locally. Confidentiality and provenance were not designed as two features that happen to coexist – withholding plaintext from the server and making the server the trustworthy record of a file's history are the same design decision, seen from two sides.

2.3 How to Test

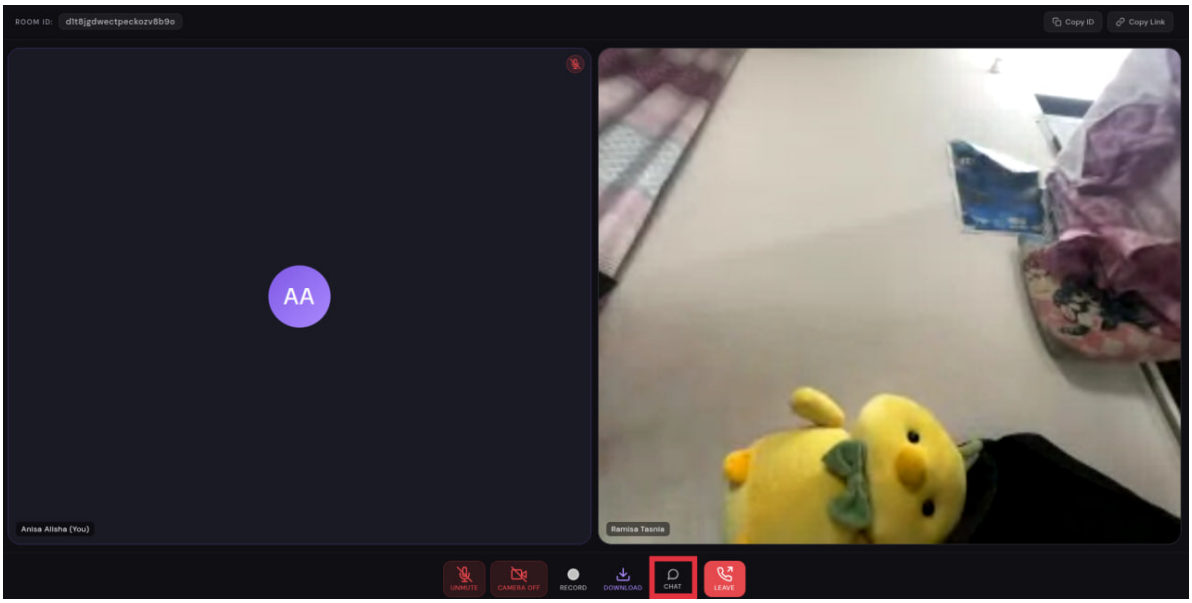
As with the audio watermarking feature, you'll need two people in the same meeting to see the everyday version of this working.

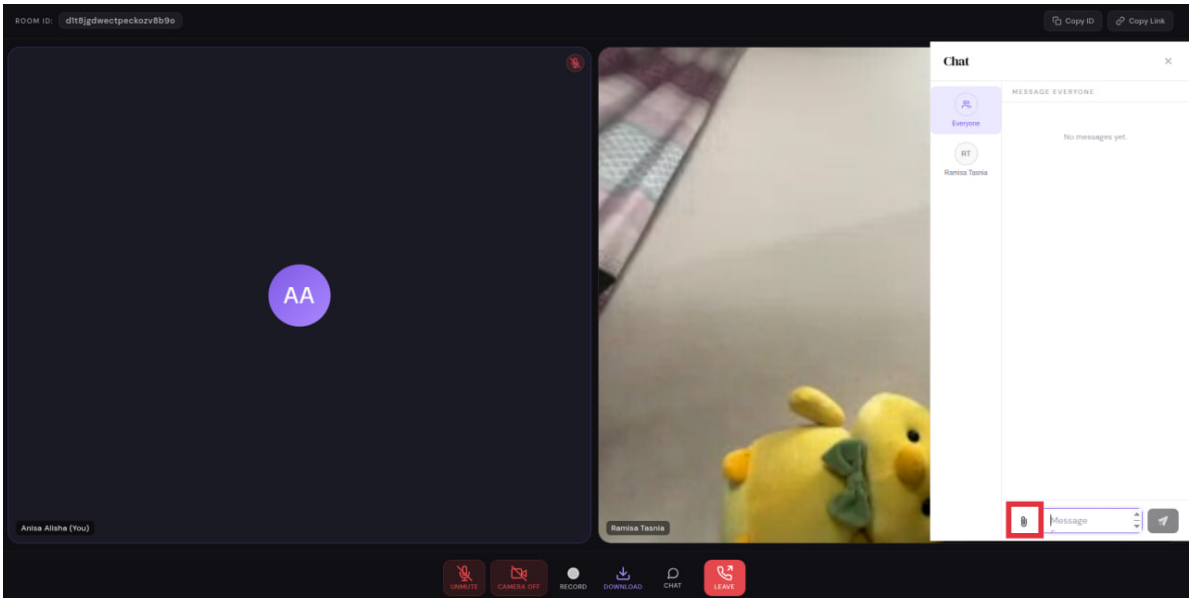
Part A – Share a file and check it arrives verified.

1. With multiple participants in the same meeting (see the audio watermarking steps for how to start one), open the in-call chat panel.

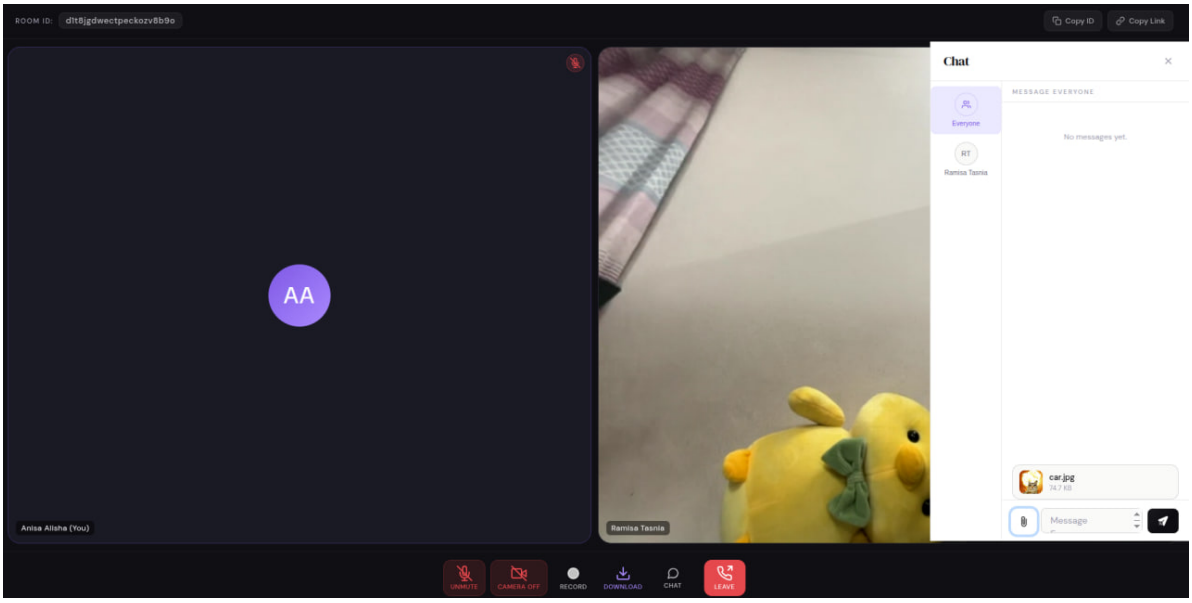


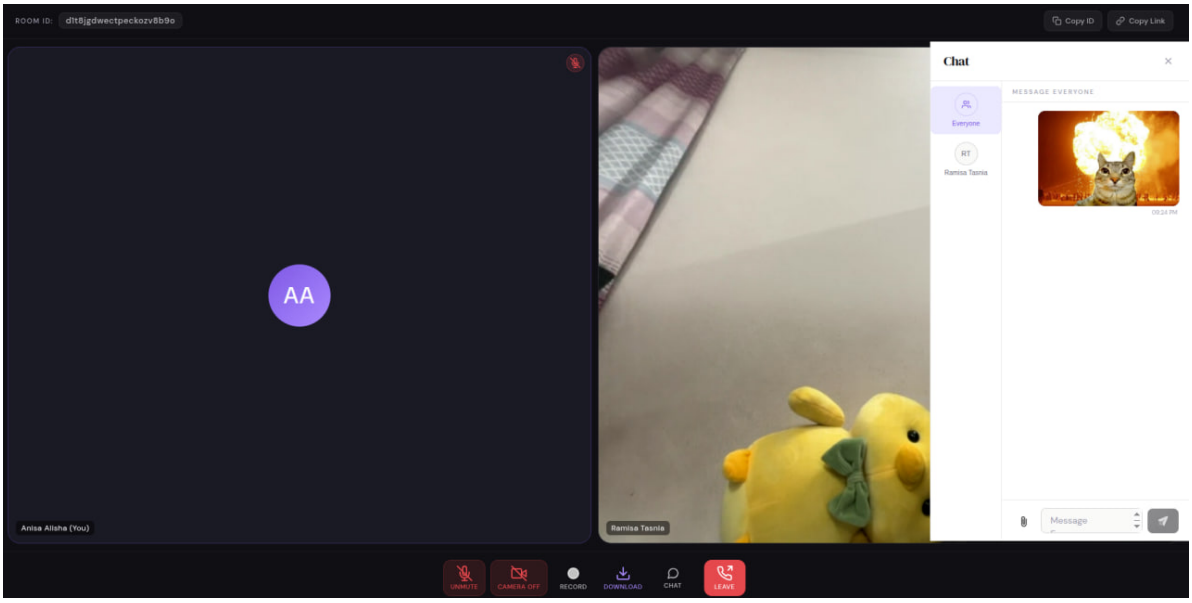
2. Click the paperclip/**Attach file** icon next to the chat box and choose any file from your computer – a photo, a PDF, anything.



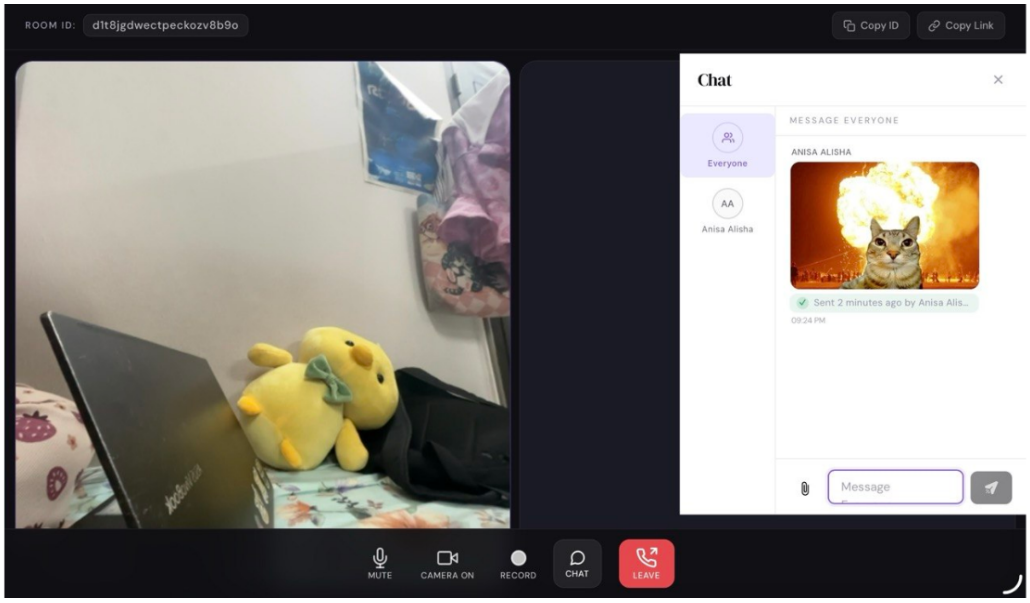

ROOM ID:
Copy ID
Copy Link
Chat
MESSAGE EVERYONE
Everyone
No messages yet.
Ramisa Tasnia
AA
Anisa Alisha (You)
Ramisa Tasnia
Message
UNMUTE
CAMERA OFF
RECORD
DOWNLOAD
CHAT
LEAVE

3. Send it.


ROOM ID:
Copy ID
Copy Link
Chat
MESSAGE EVERYONE
Everyone
No messages yet.
Ramisa Tasnia
AA
Anisa Alisha (You)
Ramisa Tasnia
car.jpg
Message
UNMUTE
CAMERA OFF
RECORD
DOWNLOAD
CHAT
LEAVE

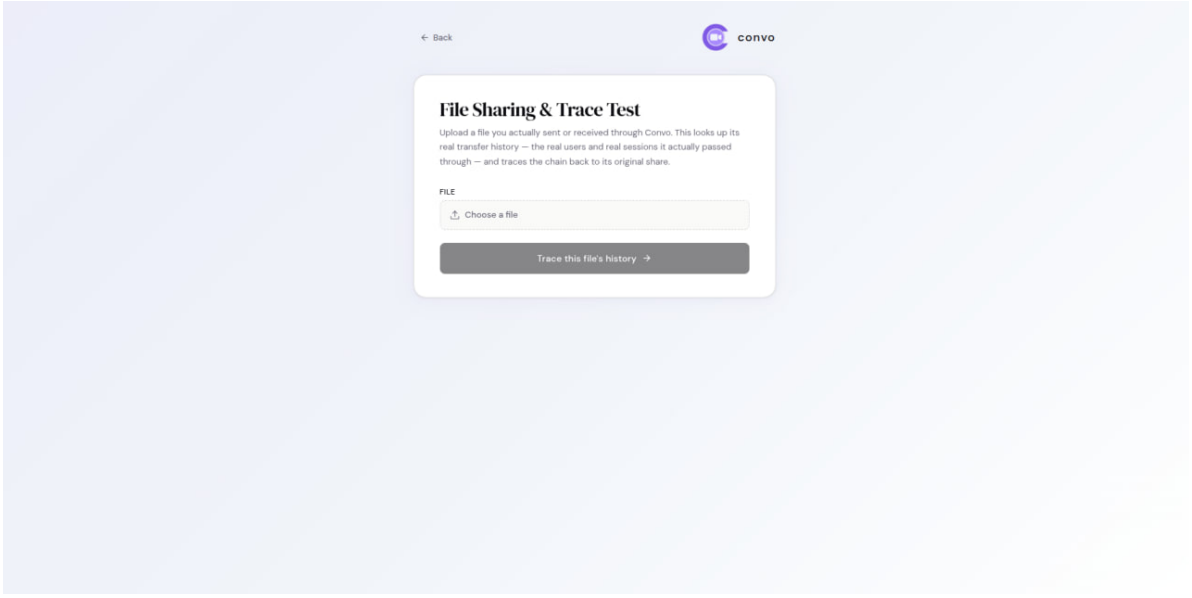


What you should see: on the *other* person's screen, the file appears in the chat with a small green checkmark line naming who sent it and when. That checkmark is the proof – it means the file arrived exactly as sent, from exactly the person it claims to be from, with nothing altered along the way.



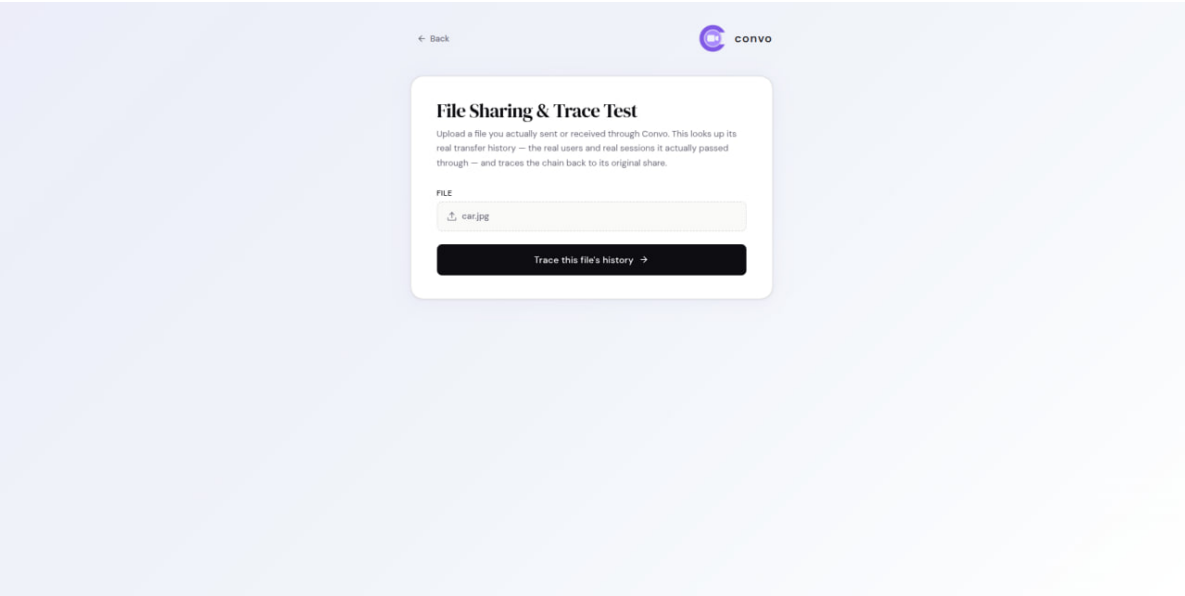
Part B – Check a file's provenance history. This test lets you see the provenance record of a file shared through Convo.

1. Open the **File Sharing Test** page: `https://convo-frontend-nine.vercel.app/file-sharing-test`



2. Upload any file from your computer.

3. Click **Trace this file's history**.



4. A timeline will appear showing the file's sharing history, including **who shared it, when it was shared, and in which meeting/session**.
5. Depending on the file's history, the trace may show:
    - verified and authorized sharing steps;
    - forwarding across different meetings/sessions;
    - the file's original share;
    - an **unauthorized** sharing step, when the sender was not a registered participant of that session; or
    - a **broken** chain when the provenance verification fails.

What you should see: a chronological timeline of the file's provenance, with each sharing step showing the relevant user, meeting/session, timestamp, and verification status.